

Méthodes d'Explicabilité en IA — Panorama V2

Du post-hoc à l'intrinsèque — Zooms SHAPformer / SoftCAM
Vers SoftCAM-Transformer (H1) pour la prévision FaaS

FOSSO Cabrel

Université de Dschang

9 mai 2026

Plan de la présentation

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ **ZOOM** SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ **ZOOM** SoftCAM (Djoumessi & Berens, 2025)

- 9 Synthèse : SHAPformer vs SoftCAM

Pourquoi expliquer ?

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ **ZOOM** SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP → CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ **ZOOM** SoftCAM (Djoumessi & Berens, 2025)

Le paradoxe performance / confiance

- Les modèles ML modernes atteignent des performances remarquables : ResNet, GPT, Transformers. . .
- Mais leur complexité les rend **opaques** :
 - Millions de paramètres
 - Décisions non traçables
 - Comportements inattendus hors distribution
- Résultat : **les utilisateurs ne font pas confiance** aux prédictions qu'ils ne comprennent pas

Dilemme

Performance élevée

vs

Confiance & déployabilité

Constat

Un modèle précis
mais inexplicable
ne peut pas être déployé
en secteur critique

Qu'est-ce qu'une bonne explication ?

3 dimensions clés

- **Portée** : locale (une prédiction) ou globale (tout le modèle)
- **Fidélité** : l'explication reflète-t-elle vraiment le raisonnement du modèle ?
- **Interprétabilité** : compréhensible par un humain non expert ?

3 propriétés théoriques (axiomes SHAP, Lundberg & Lee 2017)

- **Précision locale** : l'explication prédit correctement localement
- **Missingness** : une feature absente a contribution nulle
- **Cohérence** : si une feature contribue plus dans un modèle, son score ne diminue pas

Contexte : FAYAM et sa faille XAI

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Prédiction de charge FaaS : le problème métier

- **FaaS** (Function-as-a-Service) :
paradigme cloud où les fonctions s'exécutent à la demande
- **Défi** : invocations imprévisibles — pics, saisonnalités, profils hétérogènes
- **Enjeu** : pré-allouer les ressources pour éviter latence ou gaspillage
- **Solution** : prédire le nombre d'invocations futures à partir de l'historique

Dataset FAYAM

- Azure Functions Trace 2019
- 18 fonctions instrumentées
- 33 clusters DBSCAN (profils de charge)
- Horizon :
predictionLength minutes

Le modèle FAYAM ciblé : TimeSeriesTransformer (HuggingFace)

Architecture FAYAM Modèle 2

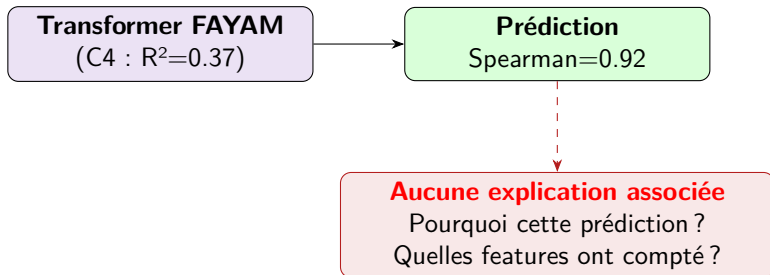
- TimeSeriesTransformerForPrediction HuggingFace
- Encodeur-Décodeur (4 couches chacun, $d_{\text{model}} = 32$)
- Attention multi-têtes, output_attentions=True disponible
- Sortie : distribution Student-t (loc, scale, df) sur 120 pas futurs

Notre baseline reproduite (Phase 1, mai 2026)

Modèle	MASE	sMAPE	RMSE	R ²	Spe
Baseline FAYAM (mixte 19 séries)	1.38	1.45	4.07	-1.26	0
Baseline FAYAM dédié C4	0.85	0.22	—	0.37	0

C4 = cluster 4 — 5 fonctions FaaS, signal 24h structuré $\Rightarrow$ **cible H1** actée.

Le verrou : prédiction sans justification



Conséquence directe

Un opérateur FaaS ne peut ni **valider**, ni **auditer**, ni **faire confiance** à un modèle qui ne justifie pas ses prédictions. **Déploiement en secteur critique : impossible**

Cadre du panorama

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 **Cadre du panorama**
- 4 Famille SHAP : panorama
- 5 ★ **ZOOM** SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP → CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ **ZOOM** SoftCAM (Djoumessi & Berens, 2025)

Comment nous allons présenter chaque méthode

Canevas en 4 points (uniforme pour toutes les méthodes)

Pour chaque méthode XAI rencontrée, nous appliquerons cette grille :

- ❶ **Contexte** de la méthode — d'où elle vient, dans quel domaine elle est née
- ❷ **Problèmes que la méthode résout** — la lacune comblée à sa création
- ❸ **Transposabilité** — est-elle applicable à d'autres domaines, notamment à la prévision FaaS ?
- ❹ **Limites** — ce qui justifiera la méthode suivante

Et entre deux méthodes : la **plus-value** apportée par la suivante.

Pourquoi ce cadre ?

Construire un **récit en chaîne** où chaque méthode comble un manque de la précédente.

Taxonomie XAI — 4 axes structurants

Axe 1 — Quand ?

- **Post-hoc** (LIME, SHAP, GradCAM...)
- **Pendant l'entraînement / intrinsèque** (SoftCAM, SHAPformer)

Axe 2 — Qui ?

- **Agnostique** (LIME, SHAP)
- **Spécifique** (CAM, SoftCAM)

Axe 3 — Portée

- Locale, semi-locale, globale

Axe 4 — Tâche

- Classification, régression, **prévision** ← **FaaS**

Famille SHAP : panorama

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Contexte

- Conférence : KDD 2016
- Domaine d'origine : **classification** (texte, images, tabulaire)
- Premier cadre XAI post-hoc *model-agnostic* rigoureux

Transposabilité

- Texte / image / tabulaire ✓
- Séries temporelles : ×
(features traitées comme indépendantes)
- Prévission : ×

Problèmes que la méthode résout

Comment expliquer la prédiction d'un modèle quelconque de façon **localement fidèle** et **interprétable** ?

⇒ apprendre un modèle linéaire sparse autour du point x .

Limites

- **Instabilité** : 2 runs = 2 explications différentes
- Pas de garantie théorique de cohérence ni missingness
- Inadapté au domaine séquentiel

Plus-value de la méthode suivante

KernelSHAP (Lundberg & Lee, 2017) apporte ce qui manque à LIME :

- **Fondement axiomatique** (théorie des jeux, Shapley 1953)
- Garantit **simultanément** précision locale, missingness, cohérence
- Solution **unique** (alors que LIME a une infinité de solutions selon le kernel)

LIME = bonne idée sans fondement ; KernelSHAP = même idée sur fondement théorique.

KernelSHAP — Lundberg & Lee (NeurIPS 2017)

Contexte

- Conférence : NeurIPS 2017
- Issu de la **théorie des jeux coopératifs** (Shapley 1953)
- Unifie LIME, DeepLIFT, SHAP linéaire sous un même cadre

Problèmes que la méthode résout

Calculer la contribution **juste et unique** de chaque feature :

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

Transposabilité

- Tabulaire / classification ✓
- Séquentiel / RNN : ✗ (ignore l'historique)
- Prédiction : ✗

Limites

- Coût exact : $2^{|F|}$ coalitions
- KernelSHAP **approxime** par échantillonnage Monte Carlo
- N'attribue à **aucun** pas temporel passé

Plus-value de la méthode suivante

TimeSHAP (Bento et al., KDD 2021) étend KernelSHAP au domaine séquentiel :

- Perturbations **bidimensionnelles** : features $\times$ événements (pas temporels)
- Élagage temporel : $O(2^I) \rightarrow O(I \cdot 2^{I-i})$
- 3 niveaux d'explication : événements / features / cellules

KernelSHAP voit le présent ; TimeSHAP voit la séquence.

TimeSHAP — Bento et al. (KDD 2021)

Contexte

- Conférence : KDD 2021 (Feedzai)
- Domaine d'origine : détection de fraude bancaire (RNN/GRU)
- Première extension SHAP au domaine séquentiel

Problèmes que la méthode résout

SHAP ignore l'historique des modèles récurrents.

TimeSHAP :

- Matrice $X \in \mathbb{R}^{d \times l}$ (d features, l événements)

Transposabilité

- Classification séquentielle ✓
- Prévion : $\times$ (conçu pour scores de proba)
- Transformer FAYAM : applicable en boîte noire mais explique mal

Limites

- **Conçu pour la classification**
- Adapté difficilement à la régression / prévision

Plus-value de la méthode suivante

WindowSHAP (Nayebi et al., 2023) attaque le coût de TimeSHAP :

- Calcul SHAP par **fenêtres temporelles** (pas par pas individuel)
- Variante *Dynamic* : pas d'hypothèse a priori sur les instants importants
- Gain CPU : -80% vs KernelSHAP sur $L = 120$

TimeSHAP attribue par pas ; WindowSHAP par fenêtre — moins fin, plus rapide.

Contexte

- Journal : Journal of Biomedical Informatics, 2023
- Domaine d'origine : médical (TBI, MIMIC-III)
- 3 datasets cliniques validés

Problèmes que la méthode résout

Réduire le coût de KernelSHAP en agrégeant les pas temporels. 3 variantes : *Stationary*, *Sliding*, *Dynamic*.

Transposabilité

- Classification clinique ✓
- Prévion : ×
- Transformer FaaS : partiellement (en boîte noire)

Limites

- Validé uniquement en **classification**
- Pas de librairie Python officielle
- Granularité grossière (fenêtres)

Plus-value de la méthode suivante

TsSHAP (Raykar et al., IBM 2023) comble le vide **prévision** :

- **Seule méthode SHAP nativement conçue pour la prévision**
- Pipeline : backtesting → features interprétables → surrogate XGBoost → TreeSHAP
- 3 portées : locale, semi-locale, globale

Avant TsSHAP : pas de SHAP pour la prévision. Avec TsSHAP : enfin un SHAP forecasting.

Contexte

- Auteurs : IBM Research, 2023
- Domaine : prévision univariée (ventes, énergie, M5)
- **Première méthode SHAP de prévision**

Transposabilité

- Prévision univariée ✓
- Multivariée : *partiel* (1 modèle par série)
- Transformer FAYAM : applicable en boîte noire
- **Notre H2 prioritaire**

Problèmes que la méthode résout

- Backtesting → prévisions historiques $\hat{y}(T + h|T)$
- Features : lags, saisonnalités, calendrier
- Surrogate XGBoost imite le

Limites

- **Univarié** (FAYAM = 18 fonctions)
- **Surrogate** XGBoost $\neq$ Transformer réel — fidélité approximative

Plus-value de la méthode suivante

SHAPformer (Hertel et al., KIT déc. 2025) apporte 3 ruptures :

- **Plus de surrogate** : SHAP exact directement sur le Transformer cible
- **Plus d'échantillonnage** : calcul exact via manipulation d'attention
- **800× plus rapide** que les approches classiques

TsSHAP imite le modèle ; SHAPformer explique le modèle réel.

Et c'est notre prochain zoom approfondi.

★ ZOOM SHAPformer (Hertel et al., 2025)

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Pourquoi un zoom approfondi sur SHAPformer ?

- **Climax du post-hoc** : la méthode la plus avancée de la famille SHAP appliquée au Transformer de prévision
- Architecture **conceptuellement proche** de notre H1 (modification d'entraînement)
- Code **public** (KIT-IAI/SHAPformer) — directement utilisable comme repli H2
- Position **intermédiaire** entre post-hoc et intrinsèque (*semi-self-explainable*)

Comprendre SHAPformer en profondeur, c'est comprendre le saut conceptuel vers SoftCAM.

SHAPformer — Canevas 4 points

Contexte

- Auteurs : Hertel, Pütz, Mikut, Hagenmeyer, Schäfer (KIT, Allemagne)
- Preprint arXiv :2512.20514, déc. 2025 (under review)
- Domaine : **prévision énergétique** (charge réseau électrique)
- Datasets : synthétique + TransnetBW (TSO allemand 2015-2019)

Problèmes que la méthode résout

- Permutation Explainer trop

Transposabilité

- Transformer ✓
- Prévision ✓
- Multivarié ✓
- **Directement applicable** au TimeSeriesTransformer FAYAM (avec réentraînement)

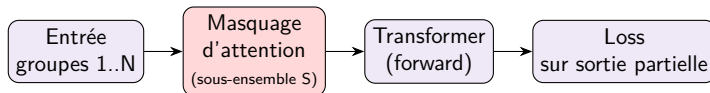
Limites

- Réentraînement requis (2–10× plus long)
- Coût d'inférence : 2^N forward passes (rapide mais

SHAPformer — Architecture (1) : phase d'entraînement

Idée centrale : entraînement avec entrées masquées

Au lieu d'entraîner sur les entrées *complètes*, SHAPformer masque aléatoirement des **groupes de features** via les poids d'attention.



Effet de cet entraînement

Le modèle apprend à prédire à **partir de n'importe quel sous-ensemble** de groupes.

Il devient *robuste aux features absents* — pré-requis pour calculer SHAP exact à l'inférence.

SHAPformer — Architecture (2) : inférence Shapley exact

Calcul SHAP sans échantillonnage

À l'inférence, on évalue le modèle sur **tous** les 2^N sous-ensembles de groupes.
Chaque évaluation = 1 forward pass rapide.

$$\phi_i = \sum_{S \subseteq V \setminus \{i\}} \frac{(N-1-|S|)! |S|!}{N!} [f(S \cup \{i\}) - f(S)] \quad (\text{SHAP exact})$$

Exemple TransnetBW

- $N = 13$ groupes (7 jours passés + 6 covariables futures)
- $2^{13} = 8\,192$ évaluations
- Temps total : **0.6 s**
- **800× plus rapide** que Permutation Explainer

Limite mathématique

Coût exponentiel en N .

Pour FAYAM (18 fonctions $\times$ lags), N peut grossir vite.

Solution proposée par les auteurs : sous-ensemble de coalitions (linéaire en N , mais perd "exactitude")

SHAPformer — Résultats clés (TransnetBW)

Performance prédictive et coût d'explication

Méthode	RMSE [MW]	Inférence / explication
Transformer baseline	263.1	0.01 s (prédiction seule)
+ Permutation Explainer	—	484 s
+ Custom Masker (Monte Carlo)	—	3.54 s
SHAPformer	265.9	0.60 s

Précision préservée ($\sim 1\%$ écart) malgré l'entraînement modifié.

Insights métier (TransnetBW)

Importance globale identifiée par SHAPformer (donnée réelle) : charge passée 47% — day-of-week 22% — hour-of-day 20% — temperature/holiday/-precipitation $< 11\%$.

Permutation Explainer sous-estime la charge passée à 10% (échantillonnage off-distribution casse la corrélation).

SHAPformer — Position : *semi-self-explainable*

Trois familles d'explicabilité — où se situe SHAPformer ?

	Modèle modifié ?	Explication = forward pass ?
Post-hoc classique (LIME, KernelSHAP, GradCAM)	non	non (calcul à part)
SHAPformer	oui (entraînement)	non (2^N passes)
SoftCAM (intrinsèque pur)	oui (architecture)	oui (1 pass)

Verdict

SHAPformer occupe une **position intermédiaire** :

- L'entraînement masqué le rend partiellement intrinsèque (le modèle est conçu pour être expliqué)
- Mais l'explication elle-même reste une phase **séparée** de la prédiction
⇒ *Semi-self-explainable*. Pas encore le saut paradigmatique.

Peut-on rendre SHAPformer pleinement self-explainable ?

Question explorée

Adapter SHAPformer pour que l'explication soit produite *en même temps* que la prédiction (1 forward pass total) ?

Pistes techniques

- 1 Tête auxiliaire qui régresse les SHAP values
(cf. **FastSHAP** – Jethani et al., NeurIPS 2021)
- 2 Distillation : modèle élève qui imite SHAPformer
- 3 Décomposition algébrique du Shapley (impossible pour Transformer non-linéaire)

Conclusion : non, sans dénaturer

- Pistes 1–2 : on perd **l'exactitude SHAP**
(devient une approximation apprise)
- Piste 3 : impossible mathématiquement
- Piste 4 : on **redescend vers SoftCAM**

Forcer SHAPformer à 1-pass =

Transition : SHAP $\rightarrow$ CAM

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Le verrou que la famille SHAP ne franchit pas

Le constat après le panorama SHAP

Toutes les méthodes SHAP — y compris SHAPformer — calculent l'explication **après** (ou en parallèle de) la prédiction.

- **Aucune** ne fait coïncider *calcul de prédiction* et *calcul d'explication*
- L'explication est toujours *ajoutée* au modèle, pas *constitutive* de lui

La rupture : famille CAM

Une autre tradition de XAI vient de la vision par ordinateur :

- **CAM** (2016) : la carte d'activation par classe est le score
- **GradCAM** (2017) : généralisation gradient, encore post-hoc
- **SoftCAM** (2025) : *intrinsèque pur* — l'explication est la prédiction

Et c'est cette voie que nous suivrons pour H1.

Famille CAM : du post-hoc à l'intrinsèque

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Contexte

- Conférence : CVPR 2016
- Domaine : **vision par ordinateur** (CNN classifieurs)
- Première méthode à *localiser* sans backprop ni perturbation

Problèmes que la méthode résout

Carte de chaleur des régions discriminantes :

$$M_c(x, y) = \sum_k w_k^c f_k(x, y)$$

Transposabilité

- CNN classifieur 2D ✓
- Séries temporelles : ✗
- Transformer : ✗ (pas de feature maps)

Limites

- Requier un GAP avant la couche FC
- Architecture imposée
- Classification uniquement

Plus-value de la méthode suivante

GradCAM (Selvaraju et al., ICCV 2017) libère CAM de la contrainte GAP :

- Utilise les **gradients** du score de classe pour pondérer les feature maps
- Applicable à **toute** architecture CNN, sans contrainte
- Compatible avec n'importe quelle couche convolutive

CAM = lisible mais contrainte ; GradCAM = lisible et libre, mais redevient post-hoc.

Contexte

- Conférence : ICCV 2017
- Domaine : vision (CNN, n'importe quelle architecture)
- Standard de facto pour visualiser CNN

Problèmes que la méthode résout

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}$$

$$L_{\text{GradCAM}}^c = \text{ReLU}(\sum_k \alpha_k^c A^k)$$

Pondération par gradient au lieu de poids FC.

Transposabilité

- Toute CNN classifieur ✓
- Régression : partiel
- Transformer : ✗

Limites

- **Post-hoc** : 1 back-pass par classe
- Résolution limitée à la feature map cible
- Sensible aux gradients saturés

Plus-value de la méthode suivante

SoftCAM (Djoumessi & Berens, mai 2025) change de paradigme :

- L'explication n'est plus *calculée* après — elle *est constitutive* du modèle
- Modification architecturale légère : conv 1×1 *class-evidence layer* + ElasticNet
- **1 forward pass** produit la prédiction *et* la carte d'évidence
- Aucun paramètre supplémentaire, aucun coût d'inférence ajouté

GradCAM explique le modèle ; SoftCAM est le modèle qui s'explique.

Et c'est notre prochain zoom approfondi.

★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Pourquoi un zoom approfondi sur SoftCAM ?

- **Climax intrinsèque** : la méthode XAI la plus pure conceptuellement (l'explication est la prédiction)
- Architecture **minimaliste** (1 couche conv 1×1 + régularisation) — *transposable*
- Validation expérimentale solide : 3 datasets médicaux, 2 backbones (VGG-16, ResNet-50)
- **Porte d'entrée explicite vers ViT/Transformer** (perspective des auteurs, p.10)
- **Inspiration directe de notre H1** (SoftCAM-Transformer)

SoftCAM — Canevas 4 points

Contexte

- Auteurs : Djoumessi & Berens (Tübingen)
- Preprint arXiv :2505.17748, mai 2025
- Domaine : **imagerie médicale** (rétine, CXR)
- 3 datasets : Kaggle DR Fundus, Retinal OCT, RSNA CXR

Transposabilité

- CNN classification 2D ✓
- ViT : *piste explicite des auteurs*
- Transformer prévision : *notre H1*
- Multivarié temporel : *à inventer*

Problèmes que la méthode résout

Les méthodes post-hoc (Grad-CAM, SHAP, IG) approximent *après coup* et sont **infidèles** au vrai raisonnement

Limites

- Validé uniquement sur **CNN imagerie**
- Trade-off λ_1, λ_2 task-specific

SoftCAM — Architecture (1) : modification minimale

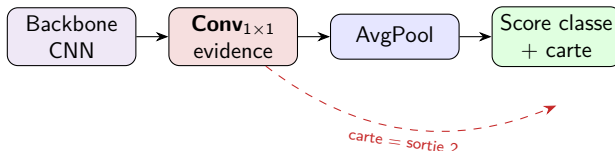
De CNN standard à SoftCAM

CNN standard :

Backbone $\rightarrow$ GAP $\rightarrow$ FCL $\rightarrow$ Softmax $\rightarrow$ classe

SoftCAM (modification minimale) :

Backbone $\rightarrow$ $\underbrace{\text{Conv}_{1 \times 1}}_{\text{evidence layer}} \rightarrow$ AvgPool $\rightarrow$ Softmax $\rightarrow$ classe



Propriété clé

- **1 forward pass** produit toutes les cartes (toutes les classes)
- Sortie : tenseur $A \in \mathbb{R}^{M \times N \times C}$ — **carte d'évidence par classe**

SoftCAM — Architecture (2) : régularisation ElasticNet

Loss augmentée (Eq. 5 de l'article)

$$\mathcal{L}(y, \hat{y}) = \underbrace{\text{CE}(y, \hat{y})}_{\text{performance}} + \underbrace{\lambda_1 \sum_{c,i,j} |A_c^{i,j}|}_{\ell_1 \text{ (Lasso)}} + \underbrace{\lambda_2 \sum_{c,i,j} \|A_c^{i,j}\|^2}_{\ell_2 \text{ (Ridge)}}$$

Sparse ($\lambda_1 > 0$)

Lasso → cartes
parcimonieuses
Régions précises

Dense ($\lambda_2 > 0$)

Ridge → cartes
lissées
Grandes régions

ElasticNet

Compromis
adaptatif
Réglable par λ_1, λ_2

Djoumessi & Berens, 2025 — $\lambda_1 \in [10^{-6}, 9 \cdot 10^{-4}]$, $\lambda_2 \in [7 \cdot 10^{-5}, 2 \cdot 10^{-4}]$

Performance préservée (Table 1)

Modèle	Backbone	AUC
Standard	VGG-16	0.938
SoftCAM dense	VGG-16	0.942
SoftCAM sparse	VGG-16	0.938
Standard	ResNet-50	0.999
SoftCAM	ResNet-50	1.000

Modification n'altère pas la précision — souvent l'améliore.

Explicabilité (Fig. 3)

Sparse SoftCAM **surpasse toutes les méthodes post-hoc** en top-k precision : vs GradCAM, IG, Guided Backprop, ScoreCAM, LayerCAM.

Coût computationnel

1 forward pass
vs 1 back-pass (gradient) ou N forward passes (perturbation) chez les concurrents

SoftCAM — Position : intrinsèque pur, perspective Transformer

Limites encore à résoudre

- Évalué uniquement sur CNN 2D
- Tâche de classification (pas de prévision)
- Cartes 16×16 grossières

Citation des auteurs (p. 10)

"In the future, we could explore the integration of SoftCAM with other standard architectures like ViT."

⇒ **Notre H1.**

Pourquoi c'est le bon point de départ pour FAYAM

- Modification architecturale **légère**
- Esprit **intrinsèque pur** (1 pass = pred + expl)
- Régularisation **transposable** au temporel
- Code de référence public
- Compatible *philosophiquement* avec un Transformer si l'on adapte la *couche cible*

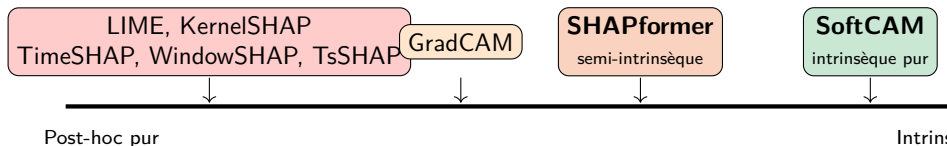
Synthèse : SHAPformer vs SoftCAM

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP → CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Tableau comparatif fin

Critère	SHAPformer	SoftCAM
Famille	SHAP	CAM
Paradigme	Semi-intrinsèque	Intrinsèque pur
Modèle modifié ?	Entraînement (masking)	Architecture (evidence layer)
Réentraînement requis ?	Oui ($2-10\times$ + long)	Oui (modification minimale)
Coût d'inférence prédictive	1 forward pass	1 forward pass
Coût d'inférence explication	2^N forward passes	0 (compris dans le 1 pass)
Type d'explication	Valeurs SHAP exactes	Carte d'évidence
Fidélité	Maximale (SHAP exact)	Maximale (par construction)
Tâche d'origine	Prévision énergie	Classification médicale
Architecture d'origine	Transformer	CNN
Validé pour FaaS multivarié ?	Direct (avec ré-entraînement)	<i>Demande adaptation</i>

L'axe semi-intrinsèque $\leftrightarrow$ intrinsèque



Le saut conceptuel manquant

- SHAPformer apporte un Transformer *pré-conditionné* pour SHAP, mais l'explication reste séparée
- SoftCAM montre qu'on peut faire *coïncider* prédiction et explication — sur CNN

Notre H1 : combiner les deux pour la prévision FaaS
⇒ SoftCAM-Transformer

★ ZOOM SoftCAM-Transformer (notre contribution H1)

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque

Motivation : la synthèse logique du parcours

Ce que les méthodes précédentes ne couvrent pas

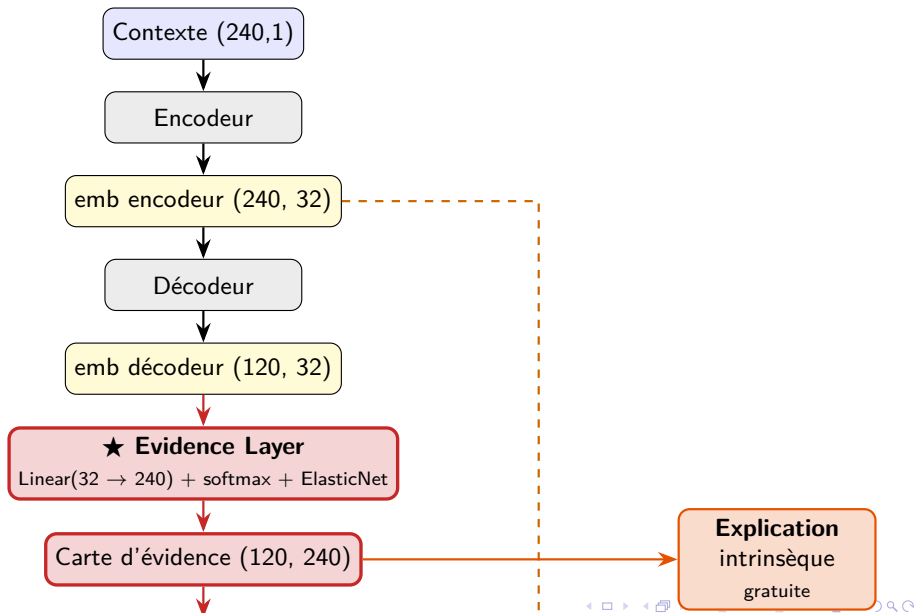
Aucune méthode existante ne couvre simultanément :

- ✓ Architecture **Transformer**
- ✓ Tâche de **prévision**
- ✓ Fidélité **intrinsèque** (1 pass)
- ✓ Séries **multivariées**
- ✓ Sans surcoût d'inférence

Notre proposition : SoftCAM-Transformer

Adapter le principe SoftCAM (evidence layer + ElasticNet) au
TimeSeriesTransformerForPrediction HuggingFace de FAYAM, cible
Cluster 4 (signal 24h structuré).

Vue d'ensemble : le flux complet



Entrées et sorties par bloc — récapitulatif

Tenseurs traversant le modèle ($d_{\text{model}} = 32$, $\text{ctx}=240$, $\text{pred}=120$)

Bloc	Entrée	Sortie	Statut
Contexte	—	(240, 1) — valeurs charge	héritage FAYAM
Encodeur	(240, 1)	(240, 32)	héritage FAYAM
emb encodeur	sortie encodeur	(240, 32) $\times 2$ (vers décodeur ET combinaison)	réutilisé
Décodeur	(240, 32) + amorce	(120, 32)	héritage FAYAM
emb décodeur	sortie décodeur	(120, 32)	transit
★ Evidence Layer	emb décodeur (120, 32)	carte (120, 240)	NOUVEAU
Carte d'évidence	sortie evidence layer	(120, 240) $\times 2$ (vers combinaison ET sortie expl)	NOUVEAU
Combinaison linéaire	carte + emb encodeur	(120, 32) — pred latent	nouveau (ca)
Projection finale	(120, 32)	(120, 3) — paramètres Student-t	héritage FAYAM
Prédiction	(120, 3)	(120,) — valeurs futures	héritage FAYAM
Explication	carte (120, 240)	(120, 240) — exposée	sortie gratuite

Lecture clé

Seul le bloc **Evidence Layer** est *vraiment nouveau*. Tout le reste est soit hérité du Transformer FAYAM (95% du modèle inchangé), soit une opération mathématique simple (combinaison linéaire).

Zoom : composition interne de l'Evidence Layer

L'Evidence Layer en 3 ingrédients

- ❶ **Une projection linéaire** : $\text{Linear}(32 \rightarrow 240)$ — apprend, pour chaque pas futur, un vecteur de 240 scores bruts sur le contexte.
- ❷ **Un softmax** (sur les 240 positions) : transforme les scores en *distribution de probabilité* positive sommant à 1.
- ❸ **Régularisation ElasticNet** dans la *loss* (pas dans la couche) : $\lambda_1 \|\text{carte}\|_1 + \lambda_2 \|\text{carte}\|_2^2$.

PyTorch — version minimaliste

```
class EvidenceLayer(nn.Module):
    def __init__(self, d_model=32, context_length=240, n_dist_params=3):
        super().__init__()
        self.evidence_proj = nn.Linear(d_model, context_length)
        self.final_proj = nn.Linear(d_model, n_dist_params)

    def forward(self, decoder_emb, encoder_emb):
        raw = self.evidence_proj(decoder_emb)           # (B, 120, 240)
        carte = torch.softmax(raw, dim=-1)              # (B, 120, 240)
        combined = torch.bmm(carte, encoder_emb)         # (B, 120, 32)
        dist_params = self.final_proj(combined)          # (B, 120, 3)
        return dist_params, carte
```

L'équation centrale

La prédiction comme combinaison interprétable

$$\text{pred_latent}[t] = \sum_{i=1}^{240} \text{carte}[t, i] \cdot \text{emb_encodeur}[i]$$

$$\hat{y}[t] = \text{Linear}_{32 \rightarrow 3}(\text{pred_latent}[t])$$

Lecture sémantique

- $\text{carte}[t, i] = \text{poids}$
d'utilisation du pas passé i
pour prédire le pas futur t
- Sommant à 1 sur les 240
positions $\Rightarrow$ *distribution*
d'attention apprise
- Formule de prédiction
lisible directement

Pourquoi c'est intrinsèque

L'explication n'est pas *calculée à part* :

elle est la **formule même** qui
produit la prédiction.

$\Rightarrow$ **Fidélité parfaite par**
construction.

Loss étendue : ElasticNet sur la carte

Loss totale

$$\mathcal{L} = \underbrace{\text{NLL}_{\text{Student-}t}(\hat{y}, y)}_{\text{prédiction (FAYAM)}} + \underbrace{\lambda_1 \| \text{carte} \|_1}_{\text{parcimonie}} + \underbrace{\lambda_2 \| \text{carte} \|_2^2}_{\text{lissage}}$$

ℓ_1 — parcimonie

Force la majorité des poids à zéro
 $\Rightarrow$ la carte ne pointe **qu'un petit nombre** de pas du contexte.
Lecture interprétable.

ℓ_2 — lissage

Évite les pics isolés bizarres $\Rightarrow$ les poids voisins varient progressivement. Cohérence temporelle.

Hyperparamètres de départ (transposés de SoftCAM CNN)

$\lambda_1 \approx 10^{-3}$; $\lambda_2 \approx 10^{-4}$ (à régler par grid search sur C4)

3 variantes de design pour l'Evidence Layer

Choix de design — point ouvert

Variante	Combinaison	Lisibilité	Expressivité
A — contexte brut	$\text{pred} = \sum_i \text{carte}[t, i] \cdot x_{\text{brut}}[i]$	maximale	limitée
B — emb encodeur	$\text{pred} = \sum_i \text{carte}[t, i] \cdot \text{emb_enc}[i] + \text{projection finale}$	élevée	élevée
C — hybride	A + biais appris (résiduel)	moyenne	moyenne

Recommandation

Variante B (carte $\times$ emb encodeur) :

- Fidèle à SoftCAM original (cartes sur feature maps, pas pixels)
- Plus expressive (capture les non-linéarités encodées)
- Moins triviale à expliquer

Risque variante A

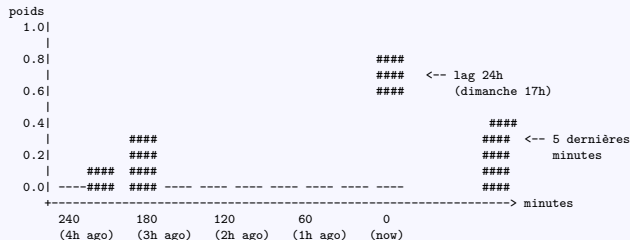
Combinaison linéaire de *valeurs brutes* :

Le modèle ne peut pas prédire un pic plus haut que le maximum observé.

Trop contraint pour C4 hétérogène (fonction 949 vs 953).

À quoi ressemble une explication — exemple sur C4

Carte d'évidence pour $t = \text{"lundi 17h00"} (function\ 953)$



Lecture

- Le modèle utilise principalement les valeurs d'il y a 24h (dimanche 17h)
- Et les 5 dernières minutes pour ajuster
- Tout le reste : poids ≈ 0 grâce à la parcimonie ElasticNet
- *Validation visuelle de l'hypothèse H1.A : la carte pointe les heures du*

Hypothèses opératoires H1.A–H1.E (déjà actées dans DECISIONS.md)

Cible H1 = Cluster 4 (5 fonctions, signal 24h)

- H1.A** La carte d'évidence pointe les heures du profil journalier (pic 17–19h, creux 2–6h)
- H1.B** Les têtes d'attention décodeur se polarisent autour des lags 1440 et 2880
- H1.C** SoftCAM ne dégrade pas la précision baseline ($R^2 \geq 0.30$, Spearman ≥ 0.85)
- H1.D** Cohérence des cartes entre les 5 fonctions du cluster (Pearson > 0.95)
- H1.E** Test sur les extrêmes : fonction 953 (best $R^2=0.60$) vs 949 (stress $R^2=0.15$)

Critère de bascule vers H2

Si après 3 semaines de prototypage SoftCAM-Transformer :

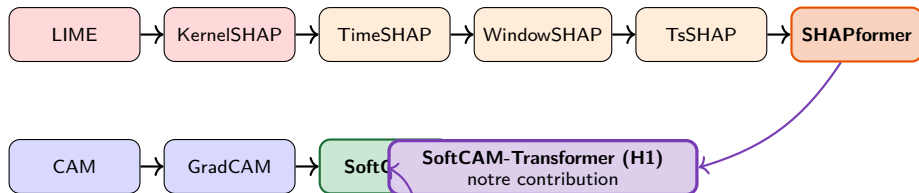
- Le modèle ne converge pas, **ou**
- H1.C est violé (dégradation $>$ seuil)

⇒ Bascule sur **H2 = SHAPformer ou TsSHAP** (repli rigoureusement justifié)

Conclusion et perspectives

- 1 Pourquoi expliquer ?
- 2 Contexte : FAYAM et sa faille XAI
- 3 Cadre du panorama
- 4 Famille SHAP : panorama
- 5 ★ ZOOM SHAPformer (Hertel et al., 2025)
- 6 Transition : SHAP $\rightarrow$ CAM
- 7 Famille CAM : du post-hoc à l'intrinsèque
- 8 ★ ZOOM SoftCAM (Djoumessi & Berens, 2025)

Récit récapitulatif — d'où vient SoftCAM-Transformer



Le récit en une phrase

SoftCAM-Transformer = la rigueur architecturale intrinsèque de SoftCAM appliquée à la cible Transformer / prévision de SHAPformer.

Perspectives — pour le chapitre Discussion

Pistes non explorées

- Vers un **SHAPformer self-explainable** ?
Discussion FastSHAP, hybridation, écueils
- Application à C0 (cas-limite, scaler interne)
- Extension multivariée : *evidence layer* sur covariables exogènes
- Cartes 2D contexte × prédiction (variante C)

Prochaines étapes Phase 2

- 1 J1 : cartographier `parameter_projection` HuggingFace
- 2 J2–J5 : prototype SoftCAMHead (variante B)
- 3 Entraînement sur C4 (5 fonctions)
- 4 Validation H1.A–H1.E
- 5 Si KO : bascule SHAPformer/TsSHAP

Calendrier

Phase 2 (H1) : 3–4 semaines $\Rightarrow$ Phase 3 (rédaction) à partir de S9

Merci pour votre attention

Des questions ?

*Cette présentation applique le canevas en 4 points
(contexte / problèmes / transposabilité / limites + plus-value de transition)
prescrit lors de la séance du 28/04/2026.*

Références I



M.T. Ribeiro, S. Singh, C. Guestrin. *“Why Should I Trust You ?” Explaining the Predictions of Any Classifier*. KDD 2016. arXiv :1602.04938.



S.M. Lundberg, S.-I. Lee. *A Unified Approach to Interpreting Model Predictions*. NeurIPS 2017.



J. Bento et al. *TimeSHAP : Explaining Recurrent Models through Sequence Perturbations*. KDD 2021. arXiv :2012.00073.



A. Nayebi et al. *WindowSHAP : An Efficient Framework for Explaining Time-Series Classifiers Based on Shapley Values*. J. Biomed. Inform., 2023.



V.C. Raykar et al. *TsSHAP : Robust Model Agnostic Feature-Based Explainability for Time Series Forecasting*. arXiv :2303.12316, IBM 2023.



M. Hertel, S. Pütz, R. Mikut, V. Hagenmeyer, B. Schäfer. *SHAPformer : Explainable Time Series Forecasting with Sampling-Free SHAP Values for Transformers*. arXiv :2512.20514, KIT, déc. 2025.



B. Zhou et al. *Learning Deep Features for Discriminative Localization*. CVPR 2016.



R.R. Selvaraju et al. *Grad-CAM : Visual Explanations from Deep Networks via Gradient-Based Localization*. ICCV 2017.



K. Djoumessi, P. Berens. *SoftCAM : Making Black Box Models Self-Explainable*. arXiv :2505.17748, mai 2025.



N. Jethani et al. *FastSHAP : Real-Time Shapley Value Estimation*. NeurIPS 2021.

Références II



J. Fayam et al. *Prédiction de charge FaaS par réseaux de neurones profonds*. Mémoire M2, Université de Dschang, 2024.